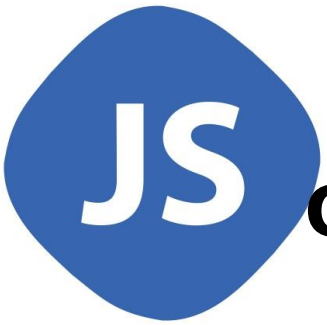


Объект XMLHTTPRequest. Введение

- Введение
- Первый запрос на XMLHttpRequest
- Тип ответа
- HTTP заголовки
- Состояние запроса
- Переадресация

Введение

Статические HTML сайты — это прошлое. На смену приходят интерактивные функциональные программы, в полной мере взаимодействующие с пользователем. Информация, полученная от сервера, практически мгновенно отображается на экране без необходимости перезагрузки страницы. Речь идет о технологии **AJAX**, что в расшифровке **Asynchronous JavaScript and XML** (Асинхронный JavaScript и XML).

AJAX — это технология динамической подгрузки данных на сайте, например для бесконечной загрузки контента на главной или другой странице.

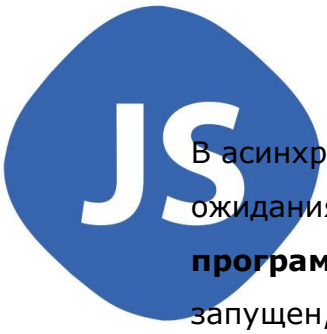
> ***Простыми словами.*** **AJAX** — обновление страницы **без ее перезагрузки**.

Характерный маркер AJAX — непрерывная подгрузка контента.

Асинхронное программирование

Асинхронное программирование — концепция программирования, при которой результат выполнения функции доступен не сразу, а через некоторое время в виде асинхронного вызова.





В асинхронном программировании длительные операции запускаются без ожидания их завершения, **не блокируя дальнейшее выполнение программы**. Это позволяет освободить поток выполнения, из которого он был запущен, что приводит к экономии ресурсов, а также предоставляет возможность параллельных вычислений.

Асинхронная обработка предполагает обработку задач независимо от основного программного потока. Это позволяет системам выполнять несколько задач одновременно, повышая эффективность и оперативность реагирования. Этот метод особенно полезен для ожидающих операций, таких как задачи ввода-вывода или **сетевые запросы**.

Асинхронное взаимодействие используется для **оптимизации высоконагруженных приложений** с частым ожиданием системы. Модели асинхронного программирования существуют во многих современных языках программирования, таких как JavaScript, Python, C# и других.

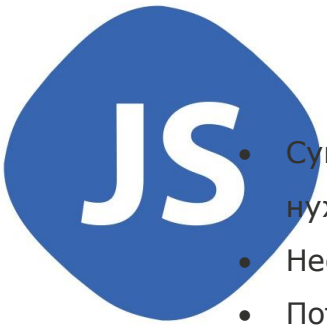
Объект XMLHttpRequest

XMLHttpRequest – это встроенный в браузер объект, который даёт возможность делать **HTTP-запросы к серверу без перезагрузки страницы**.

Несмотря на наличие слова **XML** в названии, **XMLHttpRequest** может работать с любыми данными, а не только с XML. Мы можем отправлять символьные данные, загружать/скачивать файлы, отслеживать прогресс и многое другое.

> ***Важно***. На сегодняшний день существует более современный способ выполнения асинхронных запросов - метод **fetch**. Метод подробно рассмотрим в следующих уроках.

Это не значит, что изучение объекта **XMLHttpRequest** не обязательно. Он несколько проще в понимании алгоритма асинхронного взаимодействия, кроме того, в современной веб-разработке изучение **XMLHttpRequest** целесообразно по следующим причинам:



- Существует много кода, использующего **XMLHttpRequest**, который нужно поддерживать.
- Необходимость поддерживать старые браузеры.
- Потребность в функциональности, которую **fetch** пока что не может предоставить, к примеру, отслеживание прогресса отправки на сервер.

Ну, и нельзя списывать со счетов принцип обучения – от простого к сложному. На технологии **XMLHttpRequest** разберемся, что же такое асинхронный обмен данными.

Первый запрос на XMLHttpRequest

XMLHttpRequest имеет два режима работы: **синхронный** и **асинхронный**. В большинстве случаев используется асинхронный режим.

Выполнение запроса, подразумевает следующие этапы:

1. **Создание** объекта.
2. **Инициализация** объекта.
3. **Отправка** запроса.
4. **Получение** ответа.

Создание и инициализация объекта

Синтаксис создания объекта XMLHttpRequest.

```
let xhr = new XMLHttpRequest();
```

Конструктор создаёт объект **XMLHttpRequest** (XHR). Он должен быть вызван перед обращением к любому методу класса.

Инициализация объекта:

```
xhr.open(method, URL, [async, user, password]);
```

Этот метод обычно вызывается **сразу после** конструктора **new XMLHttpRequest**.





Параметры:

- **method** – HTTP-метод. Обычно это GET или POST.
- **URL** – URL, куда отправляется запрос: строка, может быть и объект URL.
- **async** – необязательный, если указать false, тогда запрос будет выполнен синхронно.
- **user**, **password** – необязательные логин и пароль для базовой HTTP-авторизации (если требуется).

> ***Важно***. Вызов метода **open** не открывает соединение, он **конфигурирует запрос.** Непосредственно отсылается запрос только лишь после вызова **send**.

Отправка запроса

```
xhr.send([body]);
```

Параметр

- **body** – необязательный, содержит **тело запроса**.

Метод устанавливает соединение и отсылает запрос к серверу.

Некоторые типы запросов, такие как GET, не имеют тела. А некоторые, как, например, POST, используют **body** (тело запроса), чтобы **отправлять данные на сервер**.

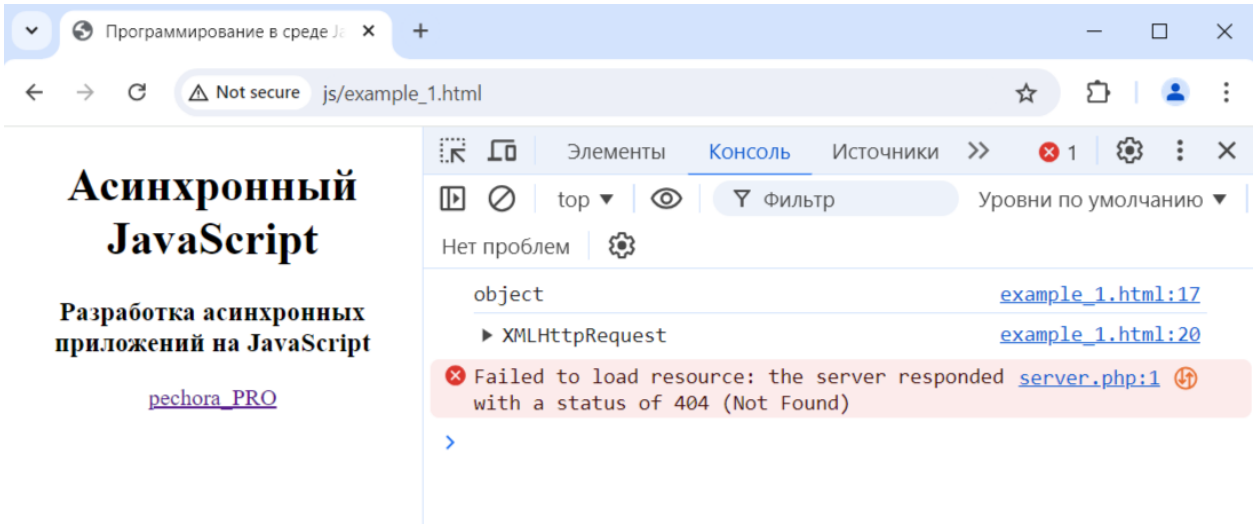
> ***Примечание***. Отправке пользовательских данных на сервер посвящен следующий урок.

При изучении асинхронных запросов обязательно анализируйте обмен заголовками в режиме разработчика (и прилагаемые скриншоты).

example_1. **index.html**

```
<script>
```

```
    // 1. создаём объект XMLHttpRequest
    let xhr = new XMLHttpRequest();
    // проверим тип переменной
    console.log(typeof xhr); // object
    // проверим объект
    console.log(xhr); // XMLHttpRequest
    // 2. инициализируем объект
    xhr.open('GET', 'http://js/server.php');
    // 3. отправка запроса
    xhr.send();
    // пока ничего не происходит,
    // просто отправлен запрос на сервер
</script>
```



Запрос отправлен на сервер, но пока ничего не происходит. Если не считать событием ошибку обращения к несуществующему файлу **server.php**.

Далее необходимо научиться получать и обрабатывать **ответ сервера**.



Получение ответа

Три наиболее используемых **события** для прослушивания **ответа сервера**:

- **onload** – происходит, когда получен какой-либо ответ, включая ответы с HTTP-ошибкой, например 404.

```
// сработает после того, как будет получен ответ сервера
xhr.onload = function() {...};
```

- **onerror** – когда запрос не может быть выполнен, например, нет соединения или невалидный URL.

```
// сработает, если запрос не может быть выполнен
xhr.onerror = function() {...}
```

- **onprogress** – происходит **периодически** во время загрузки ответа, сообщает о прогрессе.

```
// происходит периодически во время загрузки ответа
xhr.onprogress = function(event) {...};
```

После ответа сервера **результат запроса** можно получить в следующих **свойствах объекта** xhr:

- **status –** код состояния HTTP (число): 200, 404, 403 и так далее, может быть 0 в случае, если ошибка не связана с HTTP.
- **statusText** - сообщение о состоянии ответа HTTP (строка): обычно OK для 200, Not Found для 404, Forbidden для 403, и так далее.
- **response** – тело ответа сервера.

Используя событие **onerror** попробуем отловить возникшую ошибку **404 (Not Found)** и вывести соответствующее сообщение.

example_2. **index.html**

```
<script>
    // создаём объект XMLHttpRequest
```

```
    let xhr = new XMLHttpRequest();

    // вывод объекта в консоль

    console.log(xhr); // XMLHttpRequest

    // инициализируем объект

    xhr.open('GET', 'http://js/server.php');

    // отправка запроса

    xhr.send();

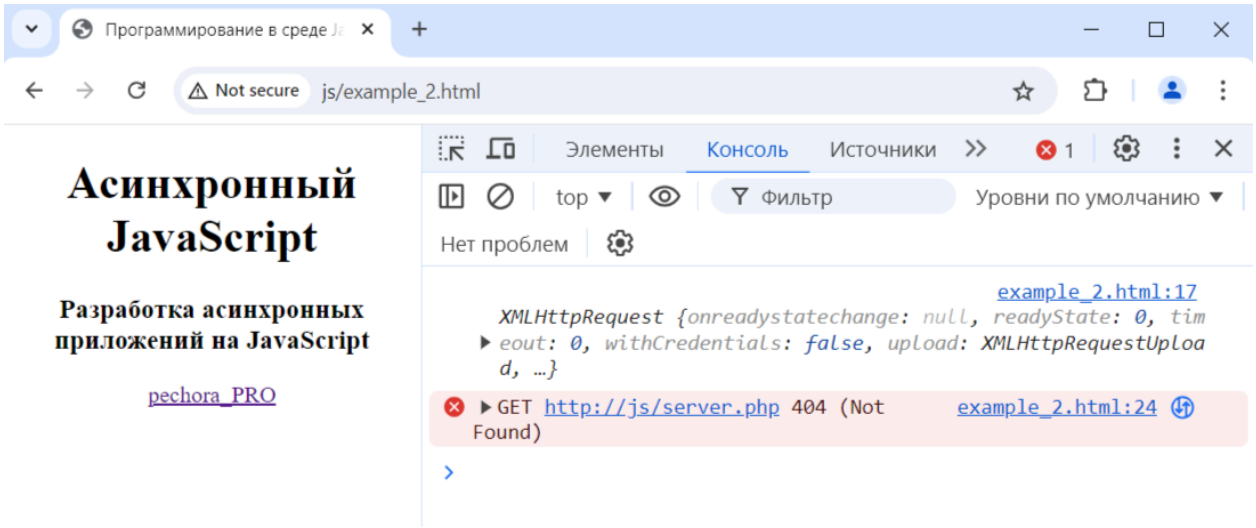
    // происходит когда запрос не может быть выполнен

    xhr.onerror = function() {

        // вывод сообщения

        alert('Ошибка выполнения запроса');

    }
</script>
```



Так не работает. Ошибка никуда не делась, но и метод **alert**() не срабатывает. Еще раз читаем определение события:

- **onerror** – срабатывает, когда запрос не может быть выполнен, например, **нет соединения** или **невалидный** URL.

Соединение есть, адрес валидный, ответ сервера пришел, все правильно, с точки зрения объекта XHR запрос успешен.

> ***Примечание***. Статус 404 (Not Found) такой же статус, как любой другой, который говорит следующее – **запрос выполнен успешно, файл, который вы запросили, не найден**.

Но раз уж мы написали событие, давайте попробуем его вызвать. Самое простое – сделать невалидным адрес URL.

```
// инициализируем объект

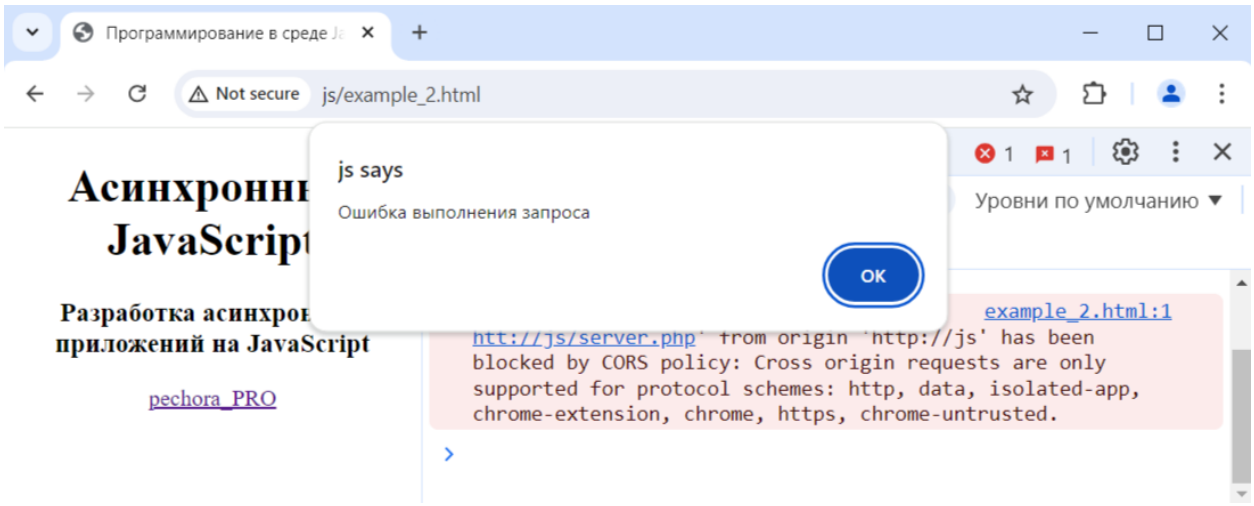
// было так

xhr.open('GET', 'http://js/server.php');

// сделаем, например, так

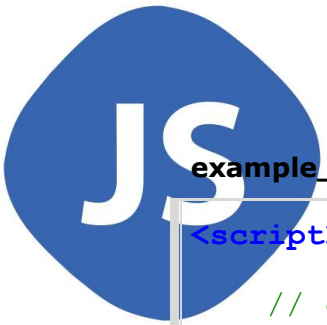
xhr.open('GET', 'htt://js/server.php');
```

Вот теперь событие **onerror** срабатывает. Результат работы скрипта представлен на рисунке.



Статус **404** с точки зрения объекта XHR не является ошибкой, а значит и обрабатывать этот статус необходимо в событии **onload**.

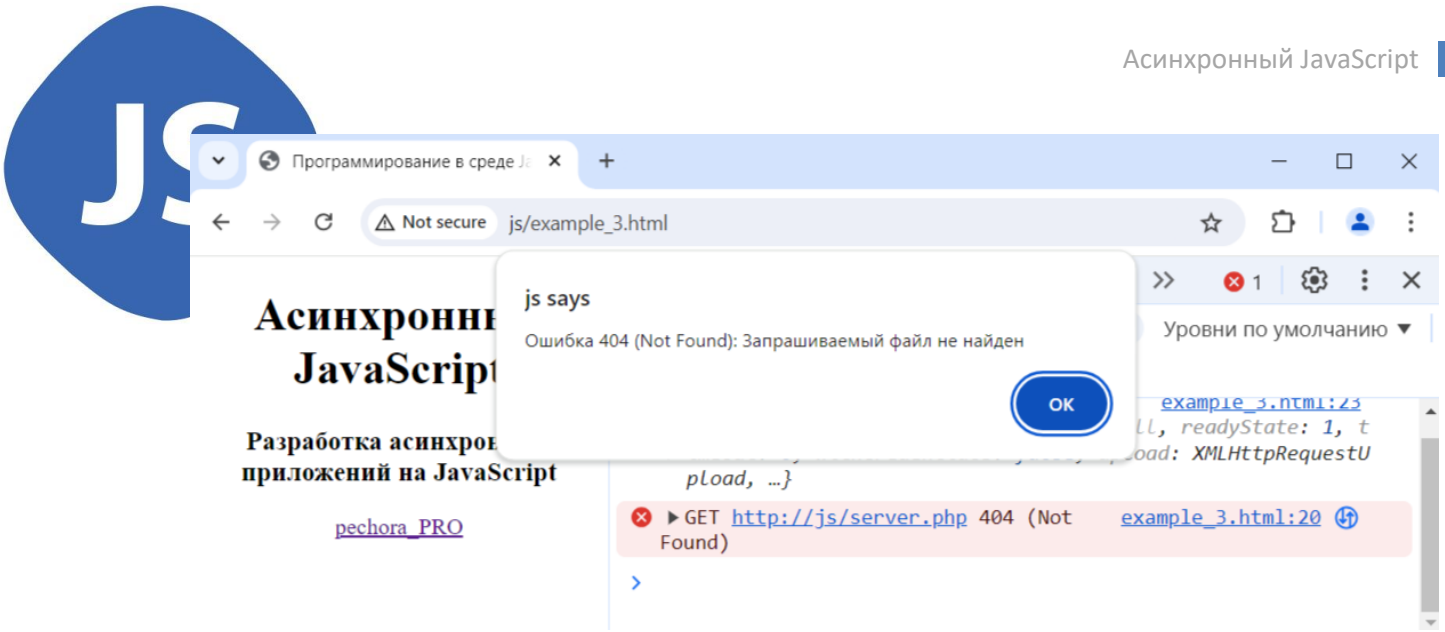
В демонстрационном примере **example_3** добавим событие **onload**.



example_3. index.html

```
<script>
    // создаём объект XMLHttpRequest
    let xhr = new XMLHttpRequest();
    // инициализируем объект
    xhr.open('GET', 'http://js/server.php');
    // отправка запроса
    xhr.send();
    // анализ объекта XMLHttpRequest
    console.log(xhr);
    // происходит когда запрос не может быть выполнен
    xhr.onerror = function(){
        // вывод сообщения
        alert('Ошибка выполнения запроса');
    };
    // сработает после получения ответа сервера
    xhr.onload = function() {
        // анализируем код HTTP-статуса ответа
        if (xhr.status == 404) {
            // 404: Not Found
            alert(`Ошибка ${xhr.status} (${xhr.statusText}):
            Запрашиваемый файл не найден `);
        }
    }
</script>
```

Результат работы скрипта представлен на скриншоте.



> ***К сведению***. И в первом, и во втором случае выходит сообщение об ошибке. Но насколько разная природа этих сообщений.

Никаких проблем не возникнет, как только по указанному адресу URL появится запрашиваемый файл (пусть даже пустой).

В директорию демонстрационного примера **example_4** добавим **файл server.php** (пустой). В функцию обработки ответа сервера – **строку** кода для вывода сообщения об успешном выполнении запроса.

example_4. index.html

```
<script>
    // создаём объект XMLHttpRequest
    let xhr = new XMLHttpRequest();
    // инициализируем объект
    xhr.open('GET', 'http://js/server.html');
    // отправка запроса
    xhr.send();
    // происходит когда запрос не может быть выполнен
    xhr.onerror = function(){
```

```
        // вывод сообщения
        alert('Ошибка выполнения запроса');
    };
    // сработает после получения ответа сервера
    xhr.onload = function() {
        // если статус HTTP ответа 404: Not Found
        if (xhr.status == 404) {
            // выводим сообщение
            alert(`Ошибка ${xhr.status} (${xhr.statusText}):
            Запрашиваемый файл не найден `);
        } else {
            alert('Запрос выполнен успешно!');
        }
    }
</script>
```

example_4. server.html

```
<!--
    этот файл здесь нужен, чтобы не было ошибки отправки запроса
    -> это клиент-сервер, детка ;)
-->
```

Получение контента

С ошибками разобрались, теперь получим контент ответа сервера. Файл **server.php** отдает строку ответа, которая будет попадать в свойство **response** объекта XHR.

example_5. index.html

```
<script>
    // создаём объект XMLHttpRequest
    let xhr = new XMLHttpRequest();
    // инициализируем объект
    xhr.open('GET', 'server.html');
    // отправка запроса
    xhr.send();
    // происходит когда запрос не может быть выполнен
    xhr.onerror = function(){
        // вывод сообщения
        alert(`Ошибка выполнения запроса`);
    };
    // сработает после получения ответа сервера
    xhr.onload = function() {
        // анализируем HTTP-статус ответа
        // если статус не 200, то произошла ошибка
        if (xhr.status != 200) {
            // выводим статус и текстовое сообщение ошибки
            alert(`Ошибка ${xhr.status}: ${xhr.statusText}`);
            // если status==200 (OK) выводим результат
        } else {
            // вывод дополнительной информации в консоль
            console.log (`Объем полученной информации -
            ${xhr.response.length} байт`);
            // в свойстве response - тело ответа сервера
```

```
            let resp = xhr.response;

            document.write("<h1>", resp, "</h1>");

        }

    };

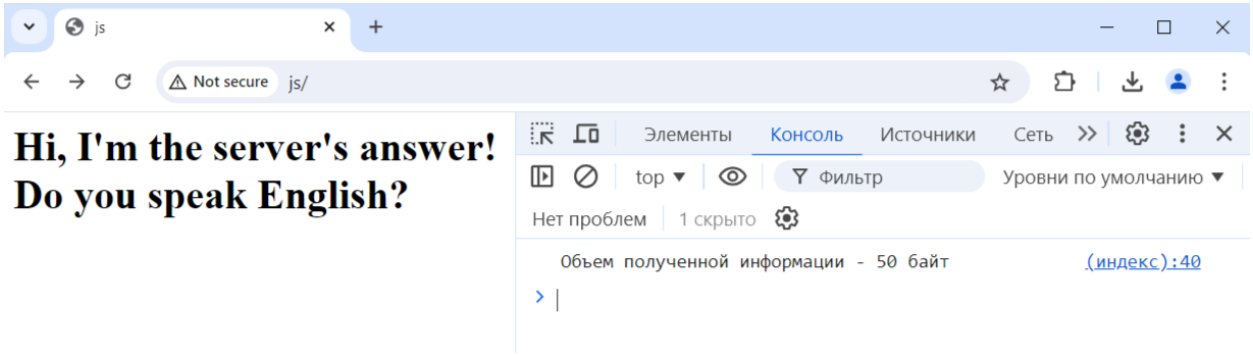
</script>
```

example_5. server.html

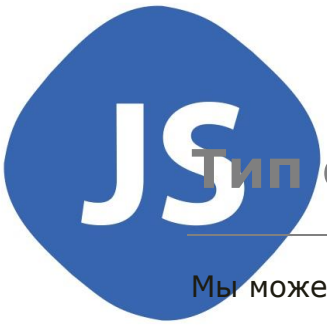
```
Hi, I'm the server's answer! Do you speak English?
```

Обратите внимание, использование асинхронной загрузки не позволяет встроить полученный контент в DOM дерева с помощью метода **document.write**(). Ответ сервера приходит после полной прорисовки страницы, поэтому вызов метода приведет к **полной замене контента** документа.

> ***К сведению***. Порядок выполнения и взаимодействие синхронных и асинхронных скриптов рассмотрим в **промисах** .



Сервер может отдавать **ответ** клиенту в разных форматах. Рассмотрим возможные варианты.



Тип ответа

Мы можем использовать свойство **responseType**, чтобы указать **ожидаемый тип ответа**:

- "" – строка (по умолчанию),
- **text** – строка,
- **json** – JSON (парсится автоматически),
- **document** – XML-документ (может использовать XPath и другие XML-методы),
- **arraybuffer** – ArrayBuffer (для бинарных данных),
- **blob** – Blob (для бинарных данных).

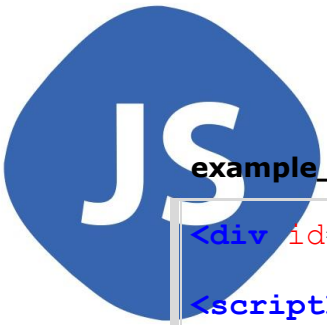
Прежде чем выполнять примеры, вы должны понять следующий **важный** нюанс.

> ***Важно***. Ключевыми моментами следующих примеров является параметр **responseType** и **формат тела** ответа сервера.
>
> - клиент **example_6.html** предупреждает (responseType), что ждет ответ в виде **строки**. Ответ приходит в виде **строки**.
> - клиент **example_7.html** предупреждает (responseType), что ждет ответ в виде **строки**. Ответ приходит в виде **JSON**. В таком случае ответ необходимо парсить в **JS-объект**.
> - клиент **example_8.html** предупреждает (responseType), что ждет ответ в виде **JSON формата**. Ответ приходит в виде **JSON**. Такой ответ **парсится автоматически**.

В демонстрационном примере **example_6** свойство:

xhr.responseType = "text"

Ответ сервера в формате **строки**. Просто выводим строку в браузер.



example_6. index.html

```
<div id="content"><!-- место для вставки контента --></div>

<script>

    // создаём объект XMLHttpRequest

    let xhr = new XMLHttpRequest();

    // инициализируем объект

    xhr.open('GET', 'server.php');

    // по умолчанию тип ответа text

    // но можно указать явно

    xhr.responseType = "text";

    // отправка запроса

    xhr.send();

    // сработает после получения ответа сервера

    xhr.onload = function() {

        // анализируем HTTP-статус ответа

        // если статус не 200, то произошла ошибка

        if (xhr.status != 200) {

            // выводим статус и текстовое сообщение ошибки

            alert(`Ошибка ${xhr.status}: ${xhr.statusText}`);

            // если status==200 (OK) выводим результат

        } else {

            // вывод дополнительной информации в консоль

            console.log (`Объем полученной информации -
            ${xhr.response.length} байт`);

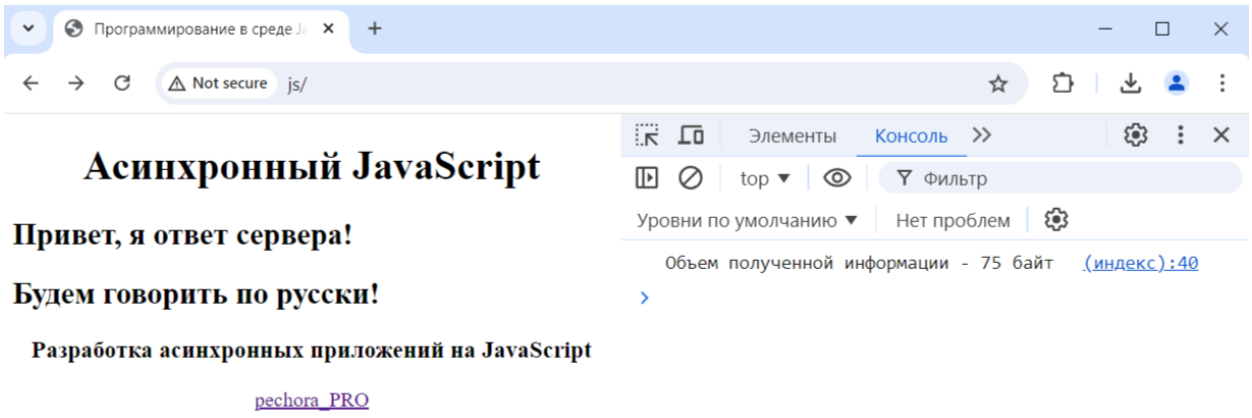
            // в свойстве response - тело ответа сервера

            let resp = xhr.response;
```

```
            // вставляем строку в документ
            document.querySelector("#content").innerHTML = resp;
        }
    };
</script>
```

example_6. server.php

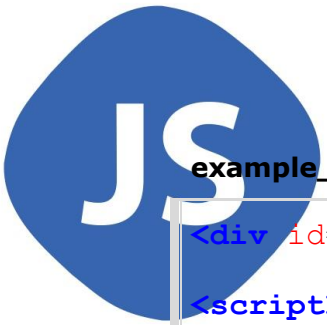
```
<?php
    // ответ сервера в серверном сценарии
    echo "
        <h2>Привет, я ответ сервера!</h2>
        <h2>Будем говорить по русски!</h2>
        ";
```



В демонстрационном примере **example_7** свойство:

xhr.responseType = "text"

Ответ сервера в формате **JSON**. Перед выводом в браузер, ответ необходимо парсить.



example_7. index.html

```
<div id="content"><!-- место для вставки контента --></div>

<script>

    // создаём объект XMLHttpRequest

    let xhr = new XMLHttpRequest();

    // инициализируем объект

    xhr.open('GET', 'server.php');

    // по умолчанию тип ответа text

    // но можно указать явно

    xhr.responseType = "text";

    // отправка запроса

    xhr.send();

    // сработает после получения ответа сервера

    xhr.onload = function() {

        // парсим ответ сервера в JS-объект

        let resp = JSON.parse(xhr.response);

        // формируем строку для вывода

        out = `

            <h3>${resp.name}</h3>

            Идентификатор альбома: ${resp.id} <br />

            Название: ${resp.name} <br />

            Дата выхода: ${resp.date} <br />

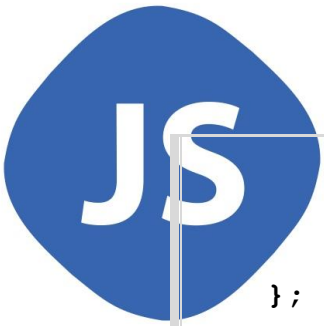
            Лейбл: ${resp.label} <br />

            Формат: ${resp.format} <br />

            Статус: ${resp.status}

        `;
```



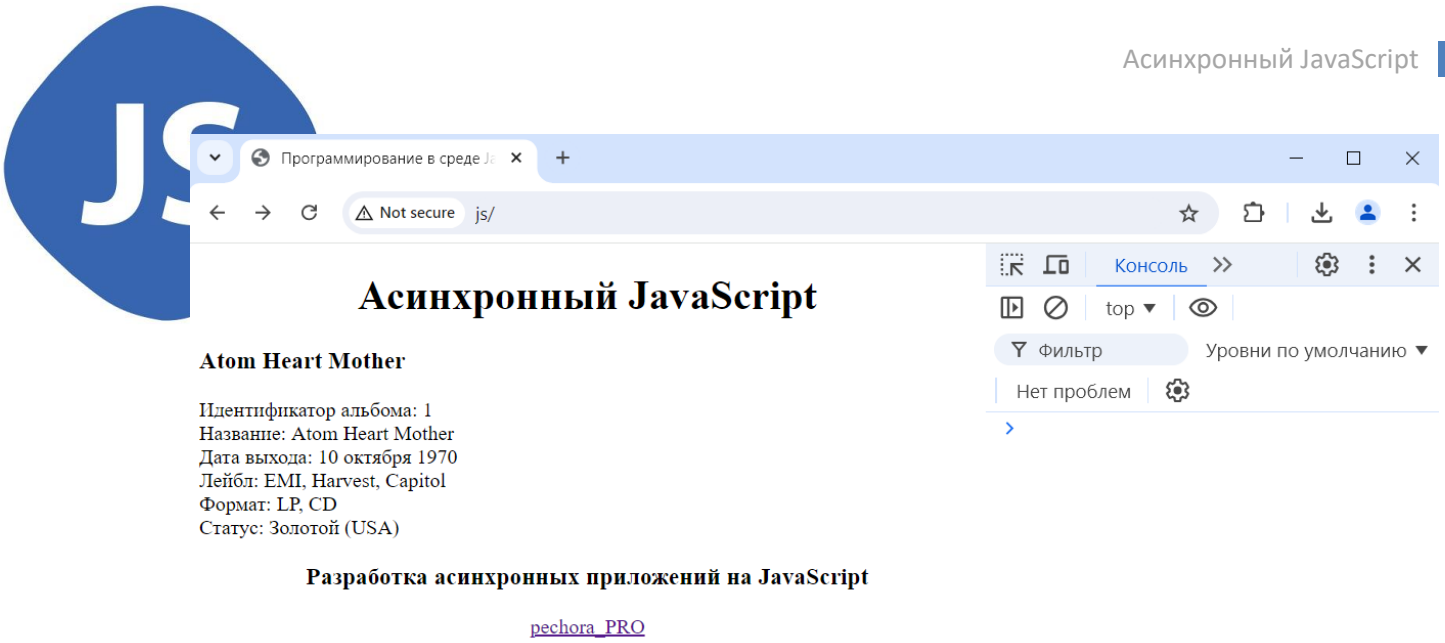


```
        // вставляем строку в документ
        document.querySelector("#content").innerHTML = out;
    };
</script>
```

example_7. server.php

```
<?php
    // ассоциативный PHP-массив
    $album =
    [
        "id" => "1",
        "name" => "Atom Heart Mother",
        "date" => "10 октября 1970",
        "label" => "EMI, Harvest, Capitol",
        "format" => "LP, CD",
        "status" => "Золотой (USA)"
    ];
    // преобразуем массив в JSON формат
    echo json_encode($album);
```





В демонстрационном примере **example_8** свойство:

xhr.responseType = "json"

Ответ сервера в формате **JSON**. Разбор строки ответа будет выполнен браузером **автоматически**.

example_8. index.html

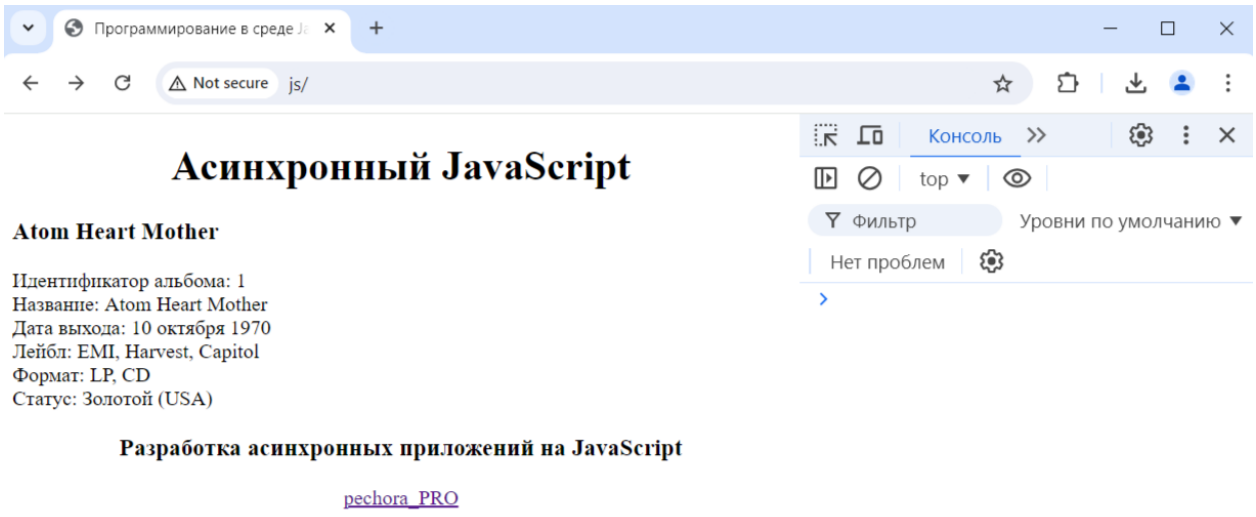
```
<div id="content"><!-- место для вставки контента --></div>
<script>
    // создаём объект XMLHttpRequest
    let xhr = new XMLHttpRequest();
    // инициализируем объект
    xhr.open('GET', 'server.php');
    // укажем ожидаемый тип ответа
    xhr.responseType = "json";
    // отправка запроса
    xhr.send();
    // сработает после получения ответа сервера
    xhr.onload = function() {
```

```
        // ответ парсится автоматически
        let resp = xhr.response;
        // формируем строку для вывода
        out = `
            <h3>${resp.name}</h3>
            Идентификатор альбома: ${resp.id} <br />
            Название: ${resp.name} <br />
            Дата выхода: ${resp.date} <br />
            Лейбл: ${resp.label} <br />
            Формат: ${resp.format} <br />
            Статус: ${resp.status}
        `;
        // вставляем строку в документ
        document.querySelector("#content").innerHTML = out;
    };
</script>
```

example_8. server.php

```
<?php
    // ассоциативный PHP-массив
    $album =
    [
        "id" => "1",
        "name" => "Atom Heart Mother",
        "date" => "10 октября 1970",
        "label" => "EMI, Harvest, Capitol",
```

```
    "format" => "LP, CD",
    "status" => "Золотой (USA)"
];
// преобразуем массив в JSON формат
echo json_encode($album);
```



Тип ответа, как правильно?

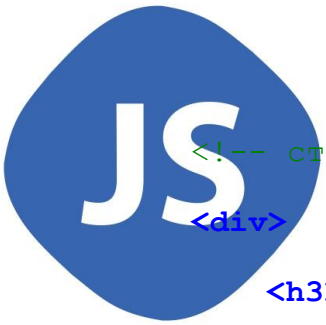
Следующий вопрос, наверное, больше интересен, для backend-разработчика. Но может и нет, асинхронный обмен стирает грань между front и back разработкой. Важно все!

Новички веб-разработки достаточно часто задаются вопросом – как правильно организовать вывод в браузер? Что должно быть в ответе сервера?

Самое верное – это ответить, да все, что угодно. Но, тем не менее...

Статичный вывод. Статичный HTML-код может легко быть ответом сервера. И часто бывает. Например, страницы сайта: **Контакты**, **О нас**, **Услуги**, **Как добраться** с большой долей вероятности будут статичными.

Пример:



```
<!-- статичный вывод -->
<div>
    <h3>Atom Heart Mother</h3>
    Идентификатор альбома: 1<br />
    Дата выхода: 10 октября 1970 <br />
    Лейбл: EMI, Harvest, Capitol<br />
    Формат: LP, CD<br />
    Статус:  Золотой (USA)
</div>
```

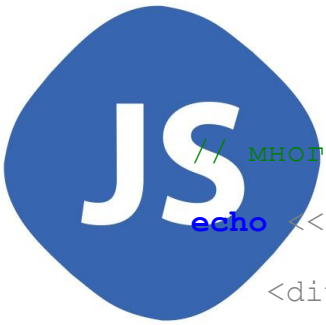
Однострочный вывод в серверном сценарии. Если вывод сервера – результат работы серверного скрипта, то скорее всего, вам в помощь оператор **echo** (в случае PHP скрипта). Или много операторов **echo**.

Пример:

```
// однострочный вывод
echo '<div>';
echo '<h3>Atom Heart Mother</h3>';
echo 'Идентификатор альбома: 1<br />';
echo 'Дата выхода: 10 октября 1970 <br />';
echo 'Лейбл: EMI, Harvest, Capitol<br />';
echo 'Формат: LP, CD<br />';
echo 'Статус:  Золотой (USA)';
echo '</div>';
```

Многострочный вывод в серверном сценарии. Если стиль вашего программирования предполагает использование многострочного оператора (что логично), используйте его.

Пример:



```
// многострочный вывод
echo <<<HERE
    <div>
    <h3>Atom Heart Mother</h3>
    Идентификатор альбома: 1<br />
    Дата выхода: 10 октября 1970 <br />
    Лейбл: EMI, Harvest, Capitol<br />
    Формат: LP, CD<br />
    Статус: Золотой (USA)
    </div>
    HERE;
```

В любом случае, как бы вы не формировали ответ, браузер построит дерево DOM, согласно поступающей на его вход **HTML разметке**.

> ***Примечание.*** Справедливости ради надо заметить, DOM будет одинаковым, а вот сама отправка данных немного отличается. Подробный анализ в демонстрационном примере **example_9**.

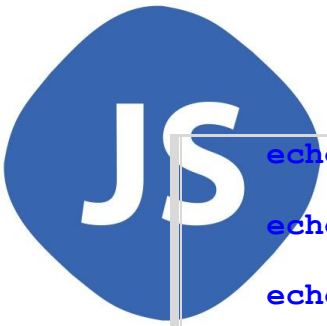
example_9. **index.html**

```
<div id="content"><!-- место для вставки контента --></div>
<script>
    // создаём объект XMLHttpRequest
    let xhr = new XMLHttpRequest();
    // инициализируем объект
    xhr.open('GET', 'server.php');
    // по умолчанию тип ответа text
    // но можно указать явно
```

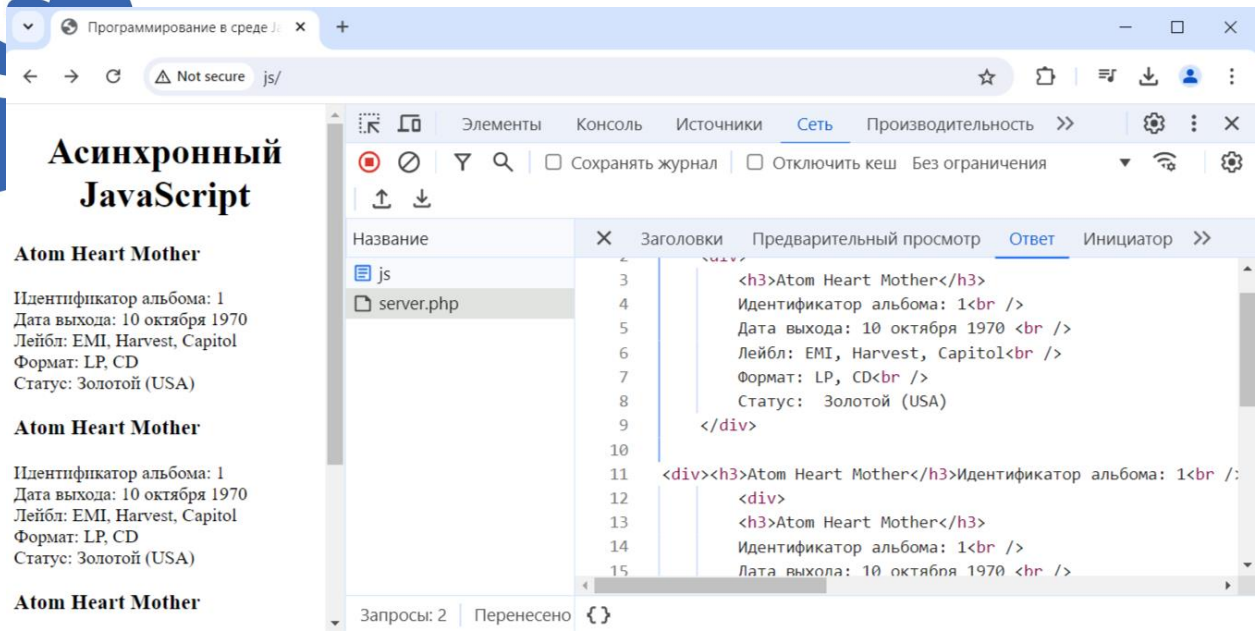
```
    xhr.responseType = "text";
    // отправка запроса
    xhr.send();
    // сработает после получения ответа сервера
    xhr.onload = function() {
        // в свойстве response - тело ответа сервера
        let resp = xhr.response;
        // вставляем строку в документ
        document.querySelector("#content").innerHTML = resp;
    }
</script>
```

example_9. server.php

```
<!-- статичный вывод -->
<div>
    <h3>Atom Heart Mother</h3>
    Идентификатор альбома: 1<br />
    Дата выхода: 10 октября 1970 <br />
    Лейбл: EMI, Harvest, Capitol<br />
    Формат: LP, CD<br />
    Статус:  Золотой (USA)
</div>
<?php
    // серверный сценарий
    // однострочный вывод
    echo '<div>';
```



```
    echo '<h3>Atom Heart Mother</h3>';
    echo 'Идентификатор альбома: 1<br />';
    echo 'Дата выхода: 10 октября 1970 <br />';
    echo 'Лейбл: EMI, Harvest, Capitol<br />';
    echo 'Формат: LP, CD<br />';
    echo 'Статус:  Золотой (USA)';
    echo '</div>';
    // многострочный вывод
    echo <<<HERE
        <div>
        <h3>Atom Heart Mother</h3>
        Идентификатор альбома: 1<br />
        Дата выхода: 10 октября 1970 <br />
        Лейбл: EMI, Harvest, Capitol<br />
        Формат: LP, CD<br />
        Статус:  Золотой (USA)
        </div>
    HERE;
?>
```

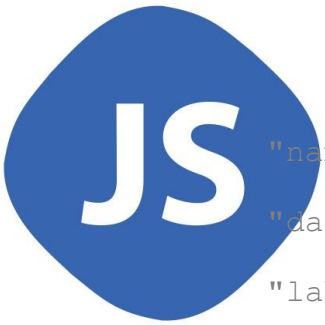


Обратите внимание, при подобном выводе вы получаете поток **текстовых символов** в переменной **response**. Дальнейшая обработка такого ответа – дело весьма затруднительное (с текстом всегда работать непросто). Поэтому, если вы захотите более гибко использовать полученные данные, ответ сервера должен быть организован в виде JSON.

JSON структура. Классическое использование асинхронных приложений подразумевает отправку данных в формате JSON.

Пример:

```
/*
    Возможные действия:
    1. подключение к СУБД
    2. извлечение данных
    3. формирование ответа
    4. отправка ответа клиенту
*/
$album =
[
    "id" => "1",
```



```
    "name" => "Atom Heart Mother",
    "date" => "10 октября 1970",
    "label" => "EMI, Harvest, Capitol",
    "format" => "LP, CD",
    "status" => "Золотой (USA)"
];
// преобразуем массив в JSON формат
echo json_encode($album);
```

Имея в JavaScript такую структуру данных, разработчик располагает большим диапазоном возможных действий по управлению элементами страницы. Именно таким образом мы будем формировать ответ при асинхронном обращении к **базам данных**.

HTTP заголовки

XMLHttpRequest умеет как указывать свои заголовки в запросе, так и читать присланные в ответ.

Для работы с HTTP-заголовками есть 3 метода:

- **setRequestHeader**(name, value)
- **getResponseHeader**(name)
- **getAllResponseHeaders**()

setRequestHeader(name, value)

Устанавливает заголовок запроса с именем **name** и значением **value**.

Пример:

```
// будем передавать данные PDF
xhr.setRequestHeader ('Content-Type', 'application/pdf');
```





Особенностью **XMLHttpRequest** является то, что отменить **setRequestHeader** невозможно. Если заголовок определён, то его нельзя снять. Повторные вызовы лишь **добавляют информацию к заголовку**, а не перезаписывают его.

example_10. **index.html**

```
<script>
    // создаём объект XMLHttpRequest
    let xhr = new XMLHttpRequest();
    // инициализируем объект
    xhr.open('GET', 'server.php');
    // 1. будем передавать данные PDF
    xhr.setRequestHeader('Content-Type', 'application/pdf');
    // 2. будем передавать данные JSON
    // xhr.setRequestHeader('Content-Type', 'application/json');
    // 3. будем передавать данные urlencoded
    // xhr.setRequestHeader('Content-Type', 'application/x-www-
    form-urlencoded');
    // отправка запроса
    xhr.send();
    // сервер вернет информацию о типе пересылаемых данных
    xhr.onload = function() {
        // вывод ответа сервера
        console.log(xhr.response);
    };
</script>
```



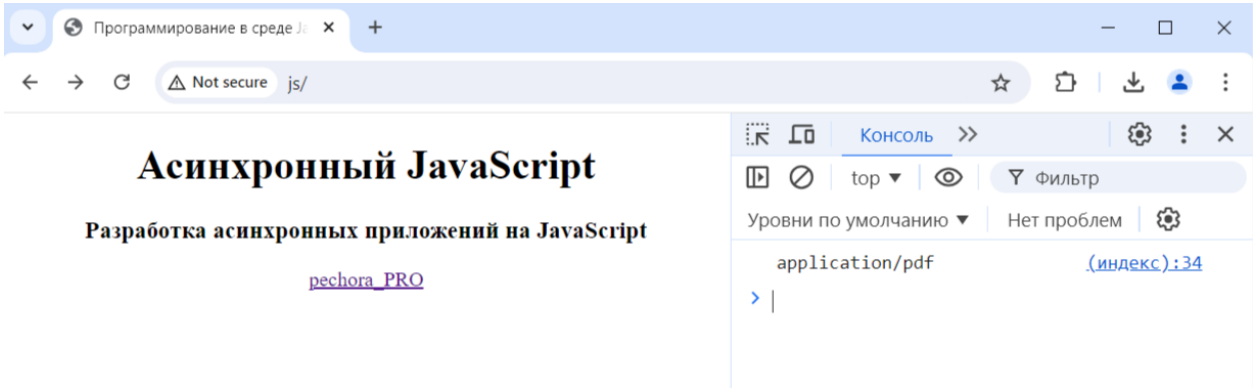
example_10. server.php

```
<?php

    $header = getallheaders();

    // сервер просто вернет тип пересылаемых данных

    echo $header['Content-Type'];
```



Некоторые заголовки управляются исключительно **браузером** (например Referer или Host), **XMLHttpRequest** не разрешено изменять их ради безопасности пользователей и для обеспечения корректности HTTP-запроса.

getResponseHeader(name)

Возвращает значение заголовка ответа сервера, указанного в аргументе **name** (кроме Set-Cookie).

Например:

```
// получим из ответа сервера формат присылаемых данных

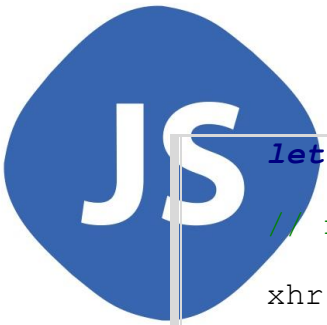
let type = xhr.getResponseHeader('Content-Type');
```

example_11. index.html

```
<div id="content"><!-- место для вставки контента --></div>

<script>

    // создаём объект XMLHttpRequest
```



```
    let xhr = new XMLHttpRequest();
    // инициализируем объект
    xhr.open('GET', 'server.php');
    // отправка запроса
    xhr.send();
    // сработает после получения ответа сервера
    xhr.onload = function() {
        // получим из ответа сервера формат присылаемых данных
        let type = xhr.getResponseHeader('Content-Type');
        // в зависимости от формата обработаем полученные данные
        if (type.includes('application/json')) {
            // если ответ - JSON строка
            // распарсим ответ и вставим данные объекта в
            выходное сообщение
            let obj = JSON.parse(xhr.response);
            out = `<h2 style="color:red">Вы зашли на сайт как
            ${obj.user} (${obj.login})</h2> `;
        } else {
            // если ответ - простая строка
            // обернем ответ тегом h2
            out = "<h2>" + xhr.response + "</h1>";
        }
        // вставляем строку в документ
        document.querySelector("#content").innerHTML = out;
    };
</script>
```

example_11. server.php

```
<?php

    // допустим, сервер может вернуть ответ в двух форматах

    // 1. в виде формата JSON строки

    header('Content-Type: application/json');

    echo '{"user": "Денис", "login":"DenisDream"}';

    // 2. в виде формата текстового сообщения

    // header('Content-Type: text/html; charset=utf-8');

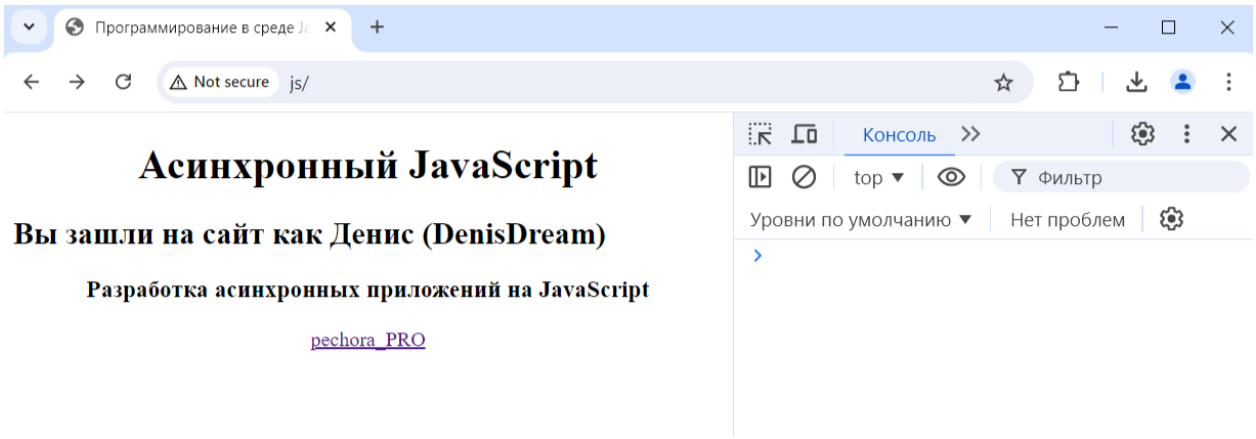
    // echo "Вы зашли на сайт как Денис (DenisDream)";

    // поэкспериментируйте с файлом сервера

    // поменяйте формат отдачи, проверьте работу клиента

    // клиент должен проанализировать формат и

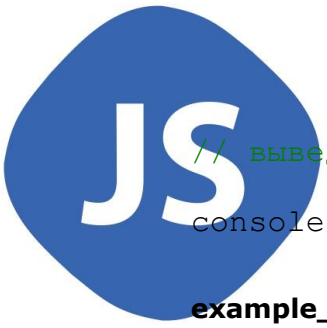
    // обработать информацию соответсвующим образом
```



getAllResponseHeaders()

Возвращает все заголовки ответа сервера (кроме Set-Cookie). Заголовки возвращаются в виде единой строки.

Например:



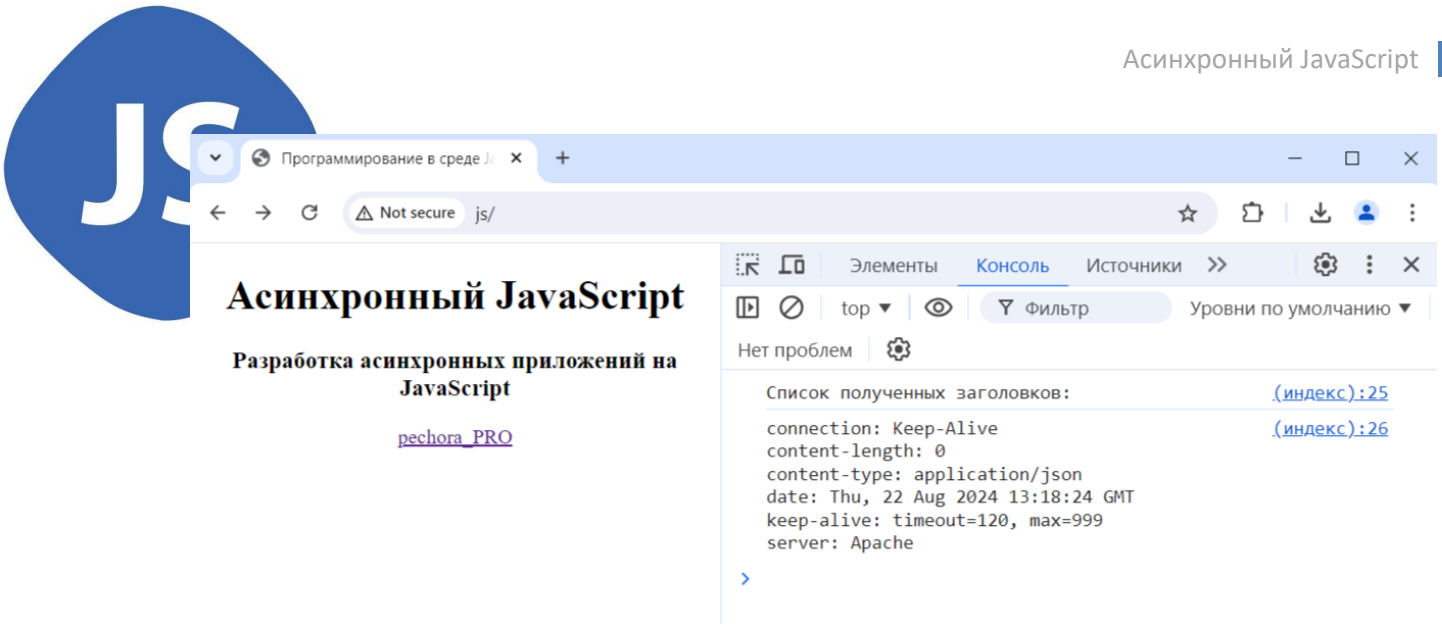
```
// выведем список полученных от сервера заголовков
console.log (xhr.getAllResponseHeaders());
```

example_12. index.html

```
<script>
    // создаём объект XMLHttpRequest
    let xhr = new XMLHttpRequest();
    // инициализируем объект
    xhr.open('GET', 'server.php');
    // отправка запроса
    xhr.send();
    // сработает после получения ответа сервера
    xhr.onload = function() {
        // выведем список полученных от сервера заголовков
        console.info ("Список полученных заголовков:");
        console.log (xhr.getAllResponseHeaders());
    };
</script>
```

example_12. server.php

```
<?php
    // сервер выдает заголовки и без нашего участия
    // но можно добавить свой
    header('Content-Type: application/json');
```



Между заголовками всегда стоит перевод строки в два символа "\r\n" (независимо от ОС), так что имея **getAllResponseHeaders**() можно легко разделить результат на отдельные заголовки. Значение заголовка всегда отделено двоеточием с пробелом ": ". Этот формат задан стандартом.

Состояние запроса

У **XMLHttpRequest** есть состояния, которые меняются по мере выполнения запроса. Текущее состояние можно посмотреть в свойстве **readyState**.

Список всех состояний, указанных в спецификации:

- **UNSENT** = 0 – исходное состояние.
- **OPENED** = 1 – вызван метод open.
- **HEADERS_RECEIVED** = 2 – получены заголовки ответа.
- **LOADING** = 3 – ответ в процессе передачи (данные частично получены).
- **DONE** = 4 – запрос завершён.

Состояния объекта **XMLHttpRequest** меняются в таком порядке:

0 → 1 → 2 → 3 → … → 3 → 4.

Состояние 3 повторяется каждый раз, когда получена часть данных.

Событие onreadystatechange()



Изменения в состоянии объекта запроса генерируют событие **onreadystatechange**.

example_13. **index.html**

```
<script>
    // создаём и инициализируем объект XMLHttpRequest
    let xhr = new XMLHttpRequest();
    xhr.open('GET', 'server.php');
    // тестируем изменения в состоянии объекта запроса
    xhr.onreadystatechange = function () {
        if (xhr.readyState == 0) {
            console.log("Исходное состояние");
        }
        if (xhr.readyState == 1) {
            console.log("Вызван метод open");
        }
        if (xhr.readyState == 2) {
            console.log("Получены заголовки ответа");
        }
        if (xhr.readyState == 3) {
            console.log("Ответ в процессе передачи (данные
            частично получены)");
        }
        if (xhr.readyState == 4) {
            console.log("Запрос завершён");
        }
    }
```

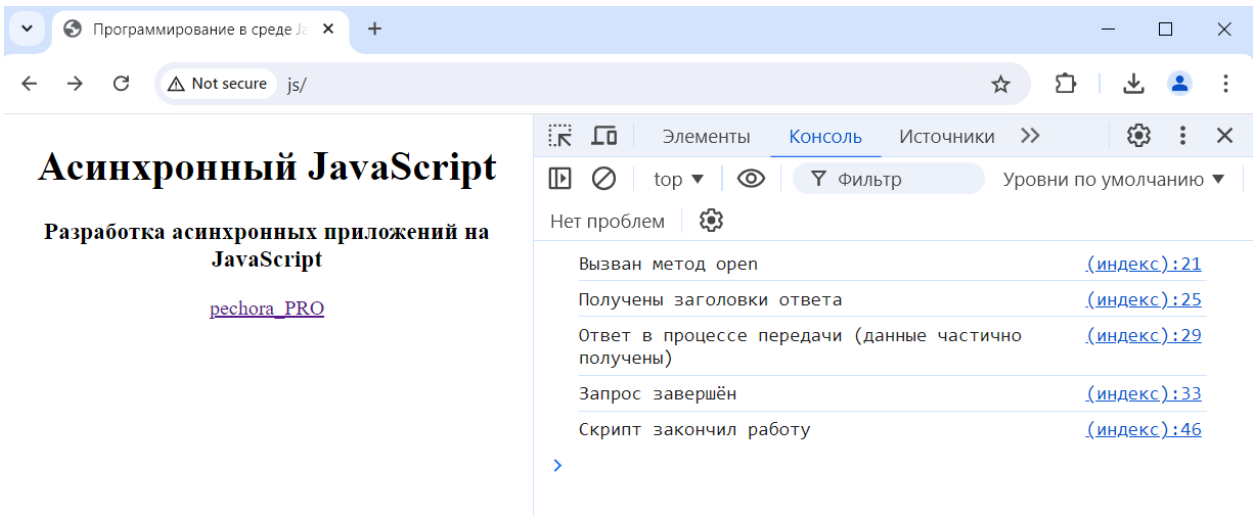


```
    // обработчик ответа сервера
    xhr.onload = function() {
        console.log("Скрипт закончил работу");
    };
    xhr.send();
</script>
```

example_13. server.php

```
<?php
    // задержка выполнения скрипта
    sleep(2);
    echo "Ответ сервера";
```

Изменения в состоянии объекта запроса продемонстрированы в следующем скриншоте.



На обработчик события **onreadystatechange** можно наткнуться в очень старом коде, так сложилось исторически, когда-то не было событий **onload** и других. Сегодня из-за существования событий **onload**/**onerror**/**onprogress** можно сказать, что событие **onreadystatechange** морально устарело.



Событие onprogress

Событие progress срабатывает только на стадии **загрузки ответа с сервера**.

А именно: если мы отправляем что-то через POST-запрос, XMLHttpRequest сперва отправит наши данные (тело запроса) на сервер, а потом загрузит ответ сервера. И событие progress будет срабатывать **только во время загрузки ответа**.

> ***Примечание.*** При работе с большими файлами нас гораздо больше интересует прогресс **отправки** данных на сервер. Событие **onprogress** тут не поможет.

example_14. **index.html**

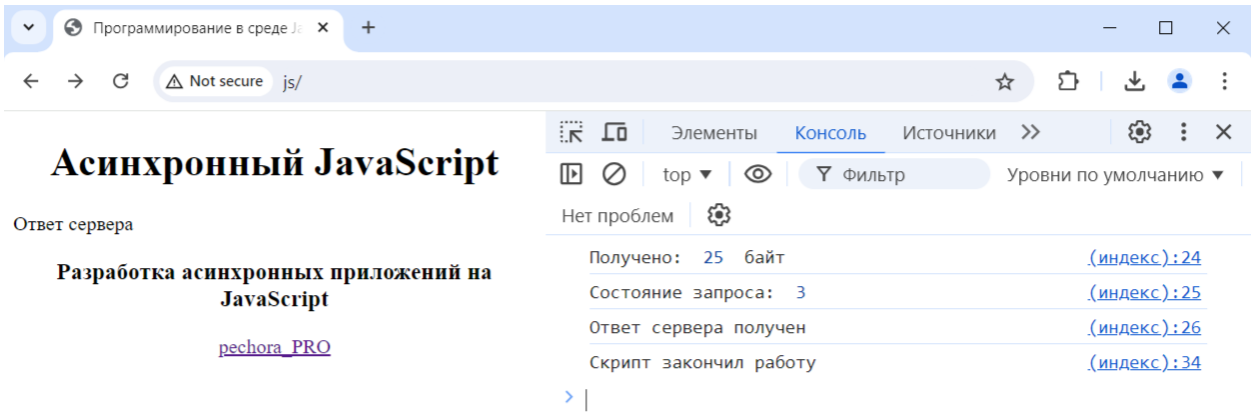
```
<div id="content"><!-- место для вставки контента --></div>
<script>
    // создаём и инициализируем объект XMLHttpRequest
    let xhr = new XMLHttpRequest();
    xhr.open('GET', 'http://js/server.php');
    // используем метод onprogress
    xhr.onprogress = function (event) {
        console.log("Получено: ", event.loaded, " байт");
        console.log("Состояние запроса: ", xhr.readyState);
        console.log("Ответ сервера получен");
    }
    // обработчик ответа сервера
    xhr.onload = function() {
        document.querySelector("#content").innerHTML =
        xhr.response;
```

```
    console.log("Скрипт закончил работу");
};

xhr.send();
</script>
```

example_14. server.php

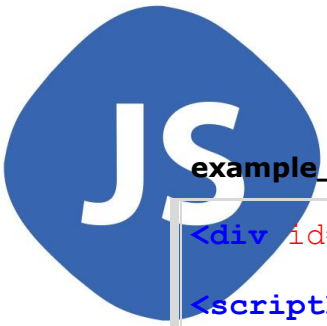
```
<?php
    // задержка выполнения скрипта
    sleep(1);
    echo "Ответ сервера"; // 25 байт
```



Существует другой объект – только для отслеживания событий отправки: **xhr.upload**. Все возможности по **отправке пользовательских данных** рассмотрим в следующем уроке.

Переадресация

В последнем разделе урока выполним асинхронную переадресацию между файлами приложения.



example_15. index.html

```
<div id="content"><!-- место для вставки контента --></div>
<script>
    // создаём и инициализируем объект XMLHttpRequest
    let xhr = new XMLHttpRequest();
    xhr.open('GET', 'http://js/server.php');
    // обработчик ответа сервера
    xhr.onload = function() {
        document.querySelector("#content").innerHTML =
        xhr.response;
    };
    xhr.send();
</script>
```

example_15. server.php

```
<?php
    // перенаправление запроса
    header("HTTP/1.1 302 Found");
    header("Location: http://js/redirect.php");
```

HTTP код перенаправления **302 Found** означает, что запрошенный ресурс был временно перемещён по адресу, указанному в заголовке **Location**.

Получив такой ответ браузер перенаправляется на новую страницу, но поисковые системы не обновляют свои ссылки на ресурс (в жаргоне SEO говорят, что вес ссылки (link-juice) не меняется и не отправляется на новый URL-адрес).

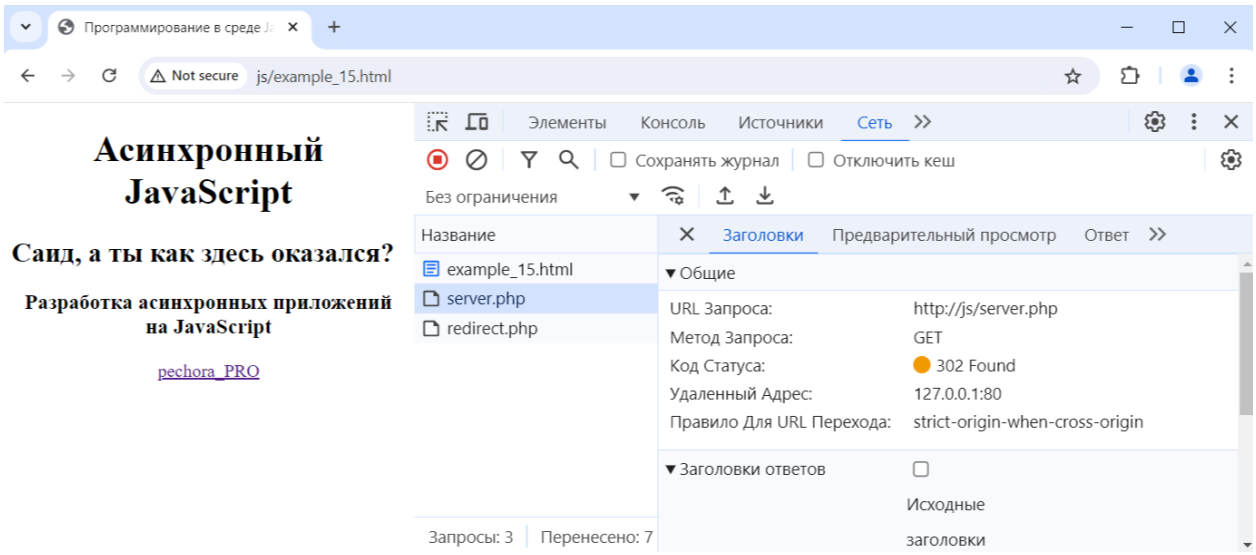


example_15. redirect.php

```
<?php

    // ответ сервера

    echo "<h2>Саид, а ты как здесь оказался?</h2>";
```



Дополнительные задания для отработки темы в **самостоятельных работах**:

- Создание сценария асинхронной загрузки контента ответа сервера.
- Создание цикла для асинхронного получения, обработки и вывода динамичного контента ответа сервера.
- Создание функции загрузки статичных страниц приложения.

P.S.

Для отработки и закрепления учебного курса **в расширенной платной версии** предоставляется следующий **раздаточный** материал.

К каждому **уроку** курса:

- Файлы **демонстрационного кода** (example).



- **Задачи** с решениями в контексте рассматриваемых вопросов урока (task).

К каждой **теме** курса:

- **Практические работы**.

К **курсу**:

- **Дополнительный практикум.**

